
PA Update Flows & Clever Variable Use

Contents

Current Process.....	3
Option 1 (Recommended): Canvas App.....	3
Architecture	3
How It Works	4
Option 2: Model-Driven App.....	5
Architecture	5
Option 3: Keep Excel but Use App Launcher	6
Flow.....	6
Option 4: Power BI Writeback (My Favorite).....	6
If I Were Building This Today	7
Clever Variable Use	7
What's Actually Happening.....	11
Common Bottlenecks.....	12
What Changes Subflow Behavior?	13
Chunking	13
Filter Before Loop	13
Replace Loops with Bulk Operations	14
Closest Thing to an Execution Plan	14
Cloud Flow.....	14
Flow Analytics	14
Run History.....	14
Compose Timers	14
Power BI Equivalent	15
Performance Analyzer.....	15
DAX Studio	15
VertiPaq Analyzer.....	15
My Rule of Thumb.....	16
Environment Variables.....	16

Datatable.....	17
PAD Equivalent.....	18
Power Apps / Power Fx Equivalent.....	18
Database Analogy	19
Loop Memory.....	20
Improve Flow w Copilot	20
Best → Worst Inputs for Copilot Analysis.....	20
What I Usually Ask For	22
Things Copilot Can Detect from JSON.....	22
Nested Loops.....	22
Missing OData Filters	23
Missing Concurrency.....	23
Excessive Variable Operations	23
"Audit Log" Type Information.....	23
Run History.....	23
Process Mining.....	24
Power Platform Admin Center.....	24
My Recommended Prompt.....	24
The App Way	25
Current State.....	25
Future State	25
User Experience	26
Replacing Excel Formulas.....	26
Power Fx.....	27
Where I Would Store the Data	27
Dataverse	27
Hybrid Approach.....	28
What I'd Build for an Enterprise Client	29
Advanced Logic & Power Fx Functions	30

Hey Co - Happy Tuesday - Power Automate - I have flow - source PBI dataset to excel w/formulas - sent as email attachment to users w ms entra id to fill out and email back. How can I do this with either model or canvas app?

Hey Tim, happy Tuesday! 😎

Yes, you can absolutely replace the "email Excel attachment → user edits → emails back" pattern with either a **Canvas App** or a **Model-Driven App**, but I'd strongly lean toward **Canvas App** for your scenario.

Current Process

```
1 Power BI Dataset
2     ↓
3 Power Automate
4     ↓
5 Excel File w/ Formulas
6     ↓
7 Email to User
8     ↓
9 User Completes
10    ↓
11 Emails Back
12    ↓
13 Manual Processing
```

This process has a few challenges:

- Version control issues
 - Email attachments everywhere
 - Users can break formulas
 - Difficult to track completion
 - Hard to enforce data validation
-

Option 1 (Recommended): Canvas App

Instead of generating Excel files, write the Power BI dataset data into a table and let users update it through a Power App.

Architecture

```
1 Power BI Dataset
2     ↓
3 Power Automate
4     ↓
5 Dataverse Table
6     ↓
7 Canvas App
8     ↓
9 User Updates Data
10    ↓
11 Dataverse
```

12 ↓
13 Approval / Reporting Flow

How It Works

1. Create Dataverse Table

Example:

Column	Type
RequestID	Text
UserUPN	Text
SalesAmount	Currency
Forecast	Number
Comments	Multiline Text
Status	Choice
DueDate	Date

2. Load Data

Power Automate:

1 Refresh Power BI Dataset
2 ↓
3 Run DAX Query
4 ↓
5 For Each Record
6 ↓
7 Upsert Dataverse Record

3. Canvas App Security

Filter records by Entra ID:

```
1 Filter(
2     ForecastTable,
3     UserUPN = User().Email
4 )
```

User automatically sees only their records.

No emails required.

4. Formula Replacement

Many Excel formulas can be recreated directly in Power Apps.

Example:

Excel:

1 =A1*B1

Canvas App:

```
1 txtQty.Value * txtPrice.Value
```

Or calculated columns in Dataverse.

5. Submit Changes

```
1 Patch(  
2     ForecastTable,  
3     ThisItem,  
4     {  
5         Forecast: Value(txtForecast.Text),  
6         Comments: txtComments.Text,  
7         Status: "Submitted"  
8     }  
9 )
```

Option 2: Model-Driven App

If the form is more data-entry oriented and less customized.

Architecture

```
1 Power BI Dataset  
2     ↓  
3 Dataverse  
4     ↓  
5 Model Driven App
```

Benefits:

- Faster to build
- Security built in
- Auditing
- Business Rules
- Approval workflows

Great when:

- Large user population
- Lots of fields
- Simple forms
- Manager approvals

Not as pretty as Canvas Apps.

Option 3: Keep Excel but Use App Launcher

If users absolutely need Excel formulas.

Flow

- | | |
|----|----------------------------|
| 1 | Power BI Dataset |
| 2 | ↓ |
| 3 | Create Excel in SharePoint |
| 4 | ↓ |
| 5 | Assign User |
| 6 | ↓ |
| 7 | Canvas App displays link |
| 8 | ↓ |
| 9 | User opens Excel Online |
| 10 | ↓ |
| 11 | Submit |

Instead of emailing files around, each user gets their own workbook stored in SharePoint.

This is a common "least disruption" upgrade.

Option 4: Power BI Writeback (My Favorite)

If this is truly a forecasting, planning, budgeting, or commentary process:

- | | |
|---|------------------|
| 1 | Power BI Dataset |
| 2 | ↓ |
| 3 | Canvas App |
| 4 | ↓ |
| 5 | Dataverse |
| 6 | ↓ |
| 7 | Power BI Report |

Embed the Canvas App directly inside Power BI.

Users:

1. Open report
2. See their numbers
3. Update forecast/comments
4. Save
5. Refresh report

No Excel at all.

If I Were Building This Today

I'd do:

1	Power BI Dataset
2	↓
3	Power Automate
4	↓
5	Dataverse
6	↓
7	Canvas App
8	↓
9	Manager Approval Flow
10	↓
11	Power BI Dashboard

Then use:

- Entra ID security
- Dataverse tables
- Canvas App forms
- Power Automate approvals
- Power BI reporting

You'll eliminate attachment emails entirely and gain real-time tracking of who has/hasn't submitted.

For your AZ-204 and Power Platform journey, this is also the most scalable enterprise pattern and aligns closely with how Microsoft recommends replacing Excel collection processes with Power Apps + Dataverse.

Clever Variable Use

In Power Automate Desktop, clever variable use centers on inline notation expressions, dynamic property expansion, smart subflow scoping, and avoiding redundant "Set variable" actions by computing values directly inside input fields. [[1](#), [2](#), [3](#), [4](#)]

Advanced Inline Expressions

- **Self-referencing math/strings:** Skip extra steps by writing expressions directly in action fields. Type or string interpolations like straight into text fields without setting intermediate variables.
- **Escaping percentage signs:** If you need an actual literal percent sign in text (like a query or URL), escape it by doubling it as so the engine does not look for a variable.
- **Dot notation for properties:** Access nested properties of complex objects or data rows instantly using dot notation (e.g., or specialized object properties).

Architecture and Scope Tricks

- **Local vs. global subflows:** Keep your main logic clean by using local subflows with scoped variables. This prevents pollution of your main global variables pane and avoids naming collisions in large projects.
- **Dynamic configuration objects:** Create a single custom JSON or Datatable variable at the start of your flow to hold environment paths, email lists, or selector flags, letting you swap configurations by changing one single source.
- **Sensitive data masking:** Right-click sensitive variables (like API keys or passwords) in the Power Automate Variable Management Panel and mark them as sensitive to mask their values from logs and run histories. [1, 8]

[Variables actions reference - Power Automate | Microsoft Learn](#)

[Work with scoped variables in Power Automate for desktop - Power Automate | Microsoft Learn](#) – local/global

Purpose & Scope

This page provides guidance on creating and managing variables in Microsoft Power Automate desktop flows, detailing various actions available for variable manipulation.

Value Proposition

Understanding how to effectively use variables enhances automation capabilities, allowing users to store, modify, and manage data efficiently within their flows.

Quick Summary of Content

- **Set Variable:** Create or overwrite a variable with a specified value.
- **Increase/Decrease Variable:** Adjust the value of numeric variables.
- **List Management:** Actions to add, remove, sort, and clear items in lists.
- **Data Table Operations:** Create, update, delete rows/columns, and manipulate data tables, including filtering and merging.
- **Random Number Generation:** Generate random numbers or lists of numbers.
- **CSV Handling:** Read from and convert data tables to CSV format.
- **JSON Conversion:** Convert between JSON strings and custom objects.
- **Error Handling:** Exceptions related to variable and data table operations are noted throughout.

[Variable data types - Power Automate | Microsoft Learn](#)

[Improve automation efficiency with scoped variables | Microsoft Learn](#)

Purpose & Scope

This page details the introduction of scoped variables in Power Automate for desktop, enhancing automation efficiency by allowing for more flexible and maintainable workflows.

Value Proposition

Scoped variables enable users to create modular workflows, making automations easier to manage and update while keeping subflows independent to avoid unexpected changes.

Quick Summary of Content

- Introduction of scoped variables for improved automation in Power Automate.
- Ability to create subflows as global or local, with local variables restricted to their subflow.
- Features include input and output variables for local subflows and a variables pane to distinguish between global and local variables.
- Links to geographic and language availability reports for this feature.

[Manage variables and the variables pane - Power Automate | Microsoft Learn](#)

Purpose & Scope

This article provides guidance on managing variables in Power Automate desktop flows, including how to create, edit, and utilize input and output variables effectively.

Value Proposition

Understanding variable management enhances automation capabilities, allowing users to manipulate data efficiently and securely within their flows.

Quick Summary of Content

- **Variables Pane:** Displays all variables, allowing users to search, rename, mark as sensitive, and filter by type.
- **Managing Flow Variables:** Inspect and control variable values, set default values, and override live values during debugging.
- **Input and Output Variables:** Create and manage variables to exchange data between cloud and desktop flows, with options for sensitivity and optionality.
- **Editing Variables During Debugging:** Modify variable values while running a flow in the debugger.
- **Known Issues:** Compatibility issues with older versions of Power Automate for desktop and limitations on sensitive variables.

[Variable manipulation and the % notation - Power Automate | Microsoft Learn](#)

Purpose & Scope

This article explains how to manipulate variables in Power Automate desktop flows, detailing how to create, use, and evaluate variables within expressions.

Value Proposition

Understanding variable manipulation enhances users' ability to store and process data effectively in their automation workflows, leading to more dynamic and efficient flows.

Quick Summary of Content

- Variables must be enclosed in percentage signs (%). Use double percentage signs (%%) to escape them when needed.
- Hardcoded values require quotes to be treated as text.
- Variables can hold blank values, which can be checked using conditional actions.
- Perform basic arithmetic operations and comparisons using standard operators.
- Use logical operators (AND, OR, NOT) for complex conditions.
- Implement string checks with functions like StartsWith, EndsWith, and Contains.
- Parentheses can alter the order of operations in expressions.

[Work with scoped variables in Power Automate for desktop - Power Automate | Microsoft Learn](#)

Purpose & Scope

This page explains the introduction of scoped variables in Power Automate for desktop, allowing users to declare variables that are accessible only within specific local subflows. This feature enhances modularity, maintainability, and debugging of desktop flows.

Value Proposition

Scoped variables improve the organization of variables, making it easier to manage and update flows, thereby enhancing overall workflow efficiency.

Quick Summary of Content

- Defines variable scope: global (accessible throughout the flow) and local (restricted to the subflow).
- Instructions for creating local subflows and variables, emphasizing that the scope cannot be changed after creation.
- Details on referencing global variables in local subflows, including the use of a global. prefix.
- Guidance on creating, passing, and retrieving input and output variables in local subflows to facilitate data exchange.
- Describes the use of the **Run subflow** action to pass values to input variables and retrieve output variables.

[Work with Power Automate desktop app - Training | Microsoft Learn](#)

[Handle variables in Power Automate desktop app - Training | Microsoft Learn](#)

- [Variables actions reference - Power Automate](#)

See all the available variables actions.

- [Text actions reference - Power Automate](#)

See all the available text actions.

- [File actions reference - Power Automate](#)

See all the available file actions.

- [Manage variables and the variables pane - Power Automate](#)

See how to manage variables and the variables pane

- [Folder actions reference - Power Automate](#)

See all the available folder actions.

- [Flow control actions reference - Power Automate](#)

Learn about the available flow control actions in desktop flows.

- [Conditional actions reference - Power Automate](#)

See all the available conditionals actions.

- [Loops actions reference - Power Automate](#)

See all the available loop actions.

Good analogy. In **Power Automate Desktop (PAD)** and cloud flows, many subflows behave like an inefficient table scan when they're invoked inside a loop.

What's Actually Happening

Imagine:

1	For Each Row in Excel
2	Call Subflow
3	Next Row

If there are 5,000 rows:

1	5,000 loop iterations
2	5,000 subflow executions

The overhead of:

- Context switching
- Variable passing
- Excel I/O
- Dataverse I/O
- API calls
- Browser actions

often dominates the actual work.

Common Bottlenecks

1. Excel Access

Worst pattern:

```
1 Loop
2   Read Excel Cell
3 Loop
```

Better:

```
1 Read Entire Worksheet
2   ↓
3 DataTable
4   ↓
5 Loop Memory Object
```

Excel open/read/write operations are among the slowest desktop actions.

2. Database Chattiness

Worst:

```
1 Loop
2   SELECT
3 Loop
```

Better:

```
1 SELECT *
2 FROM Customers
3 WHERE Status='Active'
```

once, then process locally.

Same principle as SQL optimization.

3. Subflow Call Overhead

Every subflow call has startup cost.

Bad:

```
1 Loop
2   Call Subflow
3 Loop
```

Better:

1 Pass entire DataTable

to one subflow.

4. UI Automation

Worst:

- 1 Click
- 2 Wait
- 3 Read
- 4 Click
- 5 Wait
- 6 Read

Browser and desktop UI actions are typically 10x to 100x slower than API-based operations.

AB410 and PL-500 modernization guidance generally pushes toward APIs and connectors rather than UI automation whenever possible.

What Changes Subflow Behavior?

Chunking

Instead of:

1 5000 rows

process:

1 500 row batches

or

1 1000 row batches

This reduces memory pressure and improves recovery.

Filter Before Loop

Instead of:

- 1 For Every Row
- 2 If Status = Active

do:

- 1 Get Active Rows
- 2 ↓
- 3 Loop

Same concept as SQL predicate pushdown.

Replace Loops with Bulk Operations

Example:

- | | |
|---|---------------|
| 1 | Apply to Each |
| 2 | Create Record |

versus

- | | |
|---|---------------------|
| 1 | Bulk Create Records |
|---|---------------------|

A single transaction often beats thousands of transactions.

Closest Thing to an Execution Plan

Power Automate doesn't have a true SQL-style execution plan.

However, these are the closest equivalents.

Cloud Flow

Flow Analytics

- | | |
|---|-----------|
| 1 | Flow |
| 2 | ↓ |
| 3 | Analytics |

Look at:

- Action duration
 - Success rate
 - Throughput
 - Bottlenecks
-

Run History

This is your actual execution plan.

Expand:

- | | |
|---|---------------|
| 1 | Apply to Each |
|---|---------------|

and look for:

- | | |
|---|-----------------|
| 1 | Action = 50 ms |
| 2 | Action = 8 sec |
| 3 | Action = 12 sec |

You'll normally discover one connector causing 90% of runtime.

Compose Timers

A trick I use:

1 Compose StartTime

before an action.

1 Compose EndTime

after it.

Calculate elapsed times and identify hot spots.

Power BI Equivalent

For semantic models, the "execution plan" tools are:

Performance Analyzer

Inside Power BI Desktop.

Shows:

- Visual query time
 - DAX execution time
 - Rendering time
-

DAX Studio

This is probably the nearest equivalent to SQL Server execution plans.

You can see:

- Query plans
- Storage Engine activity
- Formula Engine activity
- Server timings

SQLBI specifically recommends DAX Studio when analyzing semantic model performance. [\[sqlbi.com\]](https://sqlbi.com)

VertiPaq Analyzer

Shows:

- Cardinality
- Memory usage
- Table size
- Compression

Useful for model tuning. [\[sqlbi.com\]](https://sqlbi.com)

My Rule of Thumb

When I see:

```
1  Loop
2    Subflow
3      Loop
4        Query
```

I immediately think:

"Can this become a single query and a grouped result set?"

The biggest gains usually come from eliminating nested loops, just like replacing nested cursor logic in SQL with a set-based operation.

For someone coming from a database tuning mindset, think of PAD and Power Automate optimization as:

```
1  Filter early
2  Reduce I/O
3  Avoid nested loops
4  Batch updates
5  Cache data in memory
6  Prefer APIs over UI automation
```

Those five principles typically cut run times more than any individual PAD setting or subflow tweak.

You can use environment variables in a **Power Automate Desktop (PAD) flow** through two primary methods: **native Power Platform Environment Variables** or **Windows Environment Variables**. [[1](#), [2](#), [3](#), [4](#)]

Environment Variables

[Use environment variables in Power Platform solutions - Power Apps | Microsoft Learn](#)

[Use Power Platform environment variables in desktop flows | Microsoft Learn](#)

[Use environment variables in Power Automate solution cloud flows - Power Apps | Microsoft Learn](#)

Method 1: Native Power Platform Environment Variables

Power Automate allows you to fetch cloud-based Power Platform environment variables directly inside your desktop flow canvas without passing them manually.

1. **Find the Action:** In the PAD Actions panel, search for **Retrieve environment variable**.
2. **Select the Variable:** Drag it onto the canvas and use the dropdown to choose your variable (e.g., , , or).
3. **Use the Output:** The action automatically saves the retrieved text to an output variable called .
[[5](#), [6](#)]

Method 2: Local Windows Environment Variables

If you need to retrieve configurations specific to the machine running the automation (like specific OS file paths, temp directories, or localized parameters), use Windows Environment Variables.

- **To Read a Variable:**

1. Add the **Get Windows environment variable** action to your flow.
2. Enter the exact name of the Windows variable (e.g., , , or a custom corporate key).
3. Set the **Scope** to either **User** or **System** depending on where it is configured on the machine. [[1](#), [4](#), [7](#)]

- **To Write or Update a Variable:**

1. Add the **Set Windows environment variable** action.
2. Input the variable name, value, and scope. (Note: Writing to the "System" scope typically requires running PAD with administrator privileges). [[1](#)]

Method 3: Passing Variables via Cloud Flow (Legacy Approach)

For older setups where direct retrieval actions are unavailable or unmanaged, pass the cloud variable down into the desktop flow.

1. **In Power Automate Desktop:** Create an **Input variable** (e.g.,) in the right-side Variables pane.
2. **In Power Automate Cloud:** Add the **Run a flow built with Power Automate for desktop** action.
3. **Map the Token:** Click the field for your input parameter, open the dynamic content panel, and select your Power Platform **Environment Variable**. [[8](#), [9](#), [10](#), [11](#), [12](#)]

To help tailor this, what **type of variable** (like a URL, file path, or secret) are you trying to read, and are you triggering this flow from the **cloud**?

Datatable

No, in that example "**loop memory object**" wasn't a Power Fx function or a PAD action name. I was describing the pattern.

In PAD, the pattern is:

```
1 Launch Excel
2 ↓
3 Read from Excel worksheet
4 (Store into DataTable)
5 ↓
6 For Each
7     CurrentItem in DataTable
8 ↓
9 Process Row
```

The **DataTable** variable is the memory object.

Example:

```

1 ExcelData (DataTable)
2
3 For Each CurrentRow in ExcelData
4
5     CurrentRow["CustomerName"]
6
7 End

```

This is much faster than:

```

1 For Row 1
2     Read Cell A1
3
4 For Row 2
5     Read Cell A2
6
7 For Row 3
8     Read Cell A3

```

because PAD reads Excel once into memory and then iterates the in-memory DataTable.

PAD Equivalent

Actions:

```

1 Read from Excel worksheet
2     → Store into DataTable
3
4 For Each
5     → %CurrentRow%

```

Then reference values:

```

1 %CurrentRow["Account"]%
2 %CurrentRow["Amount"]%

```

Power Apps / Power Fx Equivalent

If you were building the same logic in a Canvas App, the equivalent "memory object" would be a **Collection**.

```

1 ClearCollect(
2     LocalData,
3     Orders
4 )

```

Then:

```

1 ForAll(

```

```

2    LocalData,
3    Patch(...)
4 )

```

Think of:

Technology	Memory Object
PAD	DataTable
Power Automate Cloud	Array Variable
Power Apps	Collection
SQL	Result Set
C#	DataTable/List

Database Analogy

What hurts performance is:

```

1  Excel
2  ↓
3  Read Row
4  ↓
5  Call Subflow
6  ↓
7  Read Row
8  ↓
9  Call Subflow

```

That's similar to a cursor.

What performs better is:

```

1  Read Entire Worksheet
2  ↓
3  DataTable in Memory
4  ↓
5  Loop DataTable
6  ↓
7  Call Subflow only when needed

```

For your PL-500/AB410 lens, I'd think of a PAD DataTable as the equivalent of loading a SQL result set into memory and processing it locally instead of repeatedly hitting the source. That's usually one of the first places I look when a desktop flow suddenly takes 20 minutes instead of 2.

Loop Memory

Improve Flow w Copilot

Yes. If your goal is "**Copilot, tell me why this flow is slow and how to improve it**", there are several levels of evidence you can provide, and some are much more useful than others.

Best → Worst Inputs for Copilot Analysis

1. Run History (Best)

The most valuable artifact is usually the **Run History details**.

Open:

1	Flow
2	→ Run History
3	→ Open a slow run

Capture:

- Duration of each action
- Apply to Each counts
- Retry counts
- Connector latency
- Failed/skipped actions
- Parallel branch timing

This is the closest thing Power Automate has to an execution plan.

For example:

1	Get Items	2 sec
2	Apply to Each	14 min
3	Update Item	2 sec x 400

Immediately tells us:

The bottleneck is the loop and connector calls.

2. Export Flow as JSON

Very useful.

Export:

1	Solutions
2	→ Export

or use:

1	Peek Code
---	-----------

for sections of the flow.

The JSON reveals:

- Nested loops
- Connector types
- Expressions
- Concurrency settings
- Trigger configuration

A lot of optimization opportunities are visible from JSON that don't show clearly in screenshots.

3. Screenshots of Flow Structure

Good for architecture review.

Example:

```
1 Trigger
2   ↓
3 Get Items
4   ↓
5 Apply to Each
6     ↓
7 Subflow
8       ↓
9 Apply to Each
```

I can often spot:

- nested loops
- unnecessary variables
- repeated queries
- serialization bottlenecks

within seconds.

4. Flow Definition Package

If exported from a Solution:

```
1 .zip
```

contains:

- flow definition
- connection references
- environment references

This is excellent for deeper review.

5. Power Automate Analytics

Capture:

1	Flow
2	→ Analytics

Especially:

- Duration trends
- Success rate
- Throughput

Useful for identifying intermittent problems.

What I Usually Ask For

For a serious performance review:

Screenshot #1

Entire flow.

1	Top-level action layout
---	-------------------------

Screenshot #2

Slowest Apply to Each.

Screenshot #3

Run History duration breakdown.

JSON

Either:

1	Peek Code
---	-----------

or

1	Export package
---	----------------

Things Copilot Can Detect from JSON

Nested Loops

1	Apply to Each
2	↓
3	Apply to Each

Classic performance killer.

Missing OData Filters

Bad:

```
1 Get Items
```

returns:

```
1 50,000 rows
```

then filters later.

Better:

```
1 Get Items
2 Filter Query:
3 Status eq 'Active'
```

Missing Concurrency

Default:

```
1 Apply to Each
```

runs sequentially.

Possible improvement:

```
1 Concurrency Control = On
2 Degree = 10
```

when connector supports it.

Excessive Variable Operations

```
1 Append Variable
2 Append Variable
3 Append Variable
```

thousands of times.

Often replaceable with:

```
1 Select
2 Filter Array
3 Join
```

which are far more efficient.

"Audit Log" Type Information

There are a few places.

Run History

Best for performance.

Shows:

- 1 Action Start
 - 2 Action End
 - 3 Duration
 - 4 Inputs
 - 5 Outputs
-

Process Mining

If licensed.

Can reveal:

- 1 Bottlenecks
- 2 Wait times
- 3 Rework

across many runs.

Power Platform Admin Center

Can expose:

- 1 Flow telemetry
- 2 Failures
- 3 Usage metrics

at scale.

My Recommended Prompt

If you're using Copilot (or me) for a flow review, provide:

- 1 Objective:
- 2 What's the flow supposed to do?
- 3
- 4 Current Runtime:
- 5 17 min
- 6
- 7 Expected Runtime:
- 8 <2 min
- 9
- 10 Issues:
- 11 Slow Apply to Each
- 12 Timeouts
- 13 SharePoint throttling
- 14
- 15 Artifacts:

- 16 1. Flow screenshot
- 17 2. Run history screenshot
- 18 3. Flow JSON (Peek Code or Export)

That combination is effectively the Power Automate equivalent of sending a DBA:

- 1 SQL Query
- 2 Execution Plan
- 3 Runtime Statistics

and gives enough information to perform a meaningful optimization review instead of generic recommendations.

The App Way

You're thinking in exactly the direction I'd recommend.

For a **real-world modernization example** that aligns with both **PL-500/PAD migration concepts** and **Applied Skills (AB410)**, I'd stop thinking of the Excel workbook as the application and start thinking of it as a temporary rules engine.

Current State

- 1 PBI Dataset
- 2 ↓
- 3 Power Automate
- 4 ↓
- 5 Excel Template
- 6 ↓
- 7 Excel Formulas Calculate Values
- 8 ↓
- 9 User Enters Data
- 10 ↓
- 11 Email Back
- 12 ↓
- 13 Flow Processes Results

The key question is:

Are the Excel formulas business logic or just calculations?

If they're calculations, Power Fx can usually replace them.

Future State

- 1 PBI Dataset
- 2 ↓
- 3 Flow
- 4 ↓
- 5 Dataverse
- 6 ↓

```
7 Canvas App
8     | Display source data
9     | Perform calculations (Power Fx)
10    | Collect user inputs
11    | Submit
12    ↓
13 Dataverse
14    ↓
15 Power Automate
16    ↓
17 Downstream Processing
```

User Experience

User signs in.

Because they're authenticated with Entra ID:

```
1 User().Email
```

the app automatically filters to their assigned records.

Example:

```
1 Filter(
2     ForecastRequests,
3     AssignedTo = User().Email
4 )
```

They see:

- Original PBI numbers
- Existing metrics
- Calculated values
- Input fields
- Submit button

No attachments.

No email responses.

No version issues.

Replacing Excel Formulas

Most formulas migrate surprisingly well.

Excel

```
1 =A1*B1
```

Power Fx

```
1 Value(txtQty.Text) * Value(txtPrice.Text)
```

Excel

```
1 =IF(A1>1000,"High","Low")
```

Power Fx

```
1 If(  
2     Amount > 1000,  
3     "High",  
4     "Low"  
5 )
```

Excel

```
1 =SUM(A1:A10)
```

Power Fx

```
1 Sum(  
2     CollectionName,  
3     Amount  
4 )
```

Excel

```
1 =VLOOKUP(...)
```

Power Fx

```
1 LookUp(  
2     ReferenceTable,  
3     ID = SelectedID  
4 )
```

Where I Would Store the Data

For AB410 training purposes I'd use:

Dataverse

Table 1

```
1 RequestHeader
```

Contains:

- RequestID
- UserUPN
- Status

- CreatedDate

Table 2

1 RequestData

Contains:

- Metric
- OriginalValue
- UserValue
- CalculatedValue

This demonstrates:

- Security
 - Relationships
 - Model-driven compatibility
 - Power Pages compatibility later
-

Hybrid Approach

If some formulas are monstrous:

1 Nested IFs
2 INDEX/MATCH
3 XLOOKUP
4 Array formulas
5 Financial formulas

don't rewrite immediately.

Instead:

1 Canvas App
2 ↓
3 Power Automate
4 ↓
5 Office Script
6 ↓
7 Excel Calculates
8 ↓
9 Return Results

This is a great interim migration pattern.

You can gradually move the formulas into Power Fx over time.

What I'd Build for an Enterprise Client

Step 1

Nightly flow:

```
1 PBI Dataset
2   ↓
3 Dataverse
```

Step 2

Canvas App:

```
1 User opens app
```

Shows:

- Assigned records
- Historical values
- Current calculations
- Editable fields

Step 3

Power Fx performs calculations live.

Example:

```
1 lblVariance.Text =
2 Text(
3     UserForecast - CurrentForecast,
4     "$#,###"
5 )
```

Step 4

Submit:

```
1 Patch(...)
```

Step 5

Power Automate triggers:

```
1 Status = Submitted
```

and performs whatever your current "email returned workbook" flow does today.

Why This Fits AB410

This demonstrates nearly every modernization concept Microsoft wants people thinking about:

- Replace file-centric automation with app-centric automation
- Move business rules from spreadsheets into Power Fx

- Use Dataverse as the system of record
- Use Entra ID for security
- Use Power Automate only for orchestration
- Eliminate human dependency on email attachments

So my recommendation is:

Keep the Power BI dataset as the source, move the workbook data into Dataverse, recreate the Excel formulas in Power Fx where practical, and let the Canvas App become the user-facing form. The downstream flow can continue almost unchanged, except it triggers from Dataverse updates instead of receiving returned Excel files. This is both a strong modernization story and a solid live demo architecture.

Advanced Logic & Power Fx Functions

. Instead of handling this logic with Power Automate actions, you create a Power Fx function by using a Dataverse data source to evaluate the customer and order criteria and email the discount code to the customer.

- [Create and use functions in Microsoft Dataverse - Power Apps | Microsoft Learn](#)
- [Implement advanced logic in Power Automate for desktop - Training | Microsoft Learn](#)
- [Work with Power Fx functions - Training | Microsoft Learn](#)
- [Exercise - Build a Power Fx Function and Integrate With a Power Automate Flow - Training | Microsoft Learn](#)

End Pfx